

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA11

COURSE OUTCOME COVERED

***CO1:** Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL3)*

Assessment Tool Weightage: 40%

Dr Gayathri A

QUESTIONS:

Total Marks:40

Annexure A

TECHNICAL PRESENTATION

Code of Conduct:

*I **D Varshini / 192320007** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

TECHNICAL PRESENTATION

1. Object-Oriented Design in Smart Healthcare Systems

A Smart Healthcare System manages patients, doctors, appointments, and medical records.

Explain how object-oriented concepts such as class, object, encapsulation, inheritance, and polymorphism are applied in this system. Analyze how these concepts improve scalability, security, and maintainability.

Answer:

A Smart Healthcare System manages patients, doctors, appointments, medical records, prescriptions, and hospital services. Object-oriented programming helps represent these real-world healthcare entities as software components. The main object-oriented concepts used are class, object, encapsulation, inheritance, and polymorphism.

1. Class

A class is a blueprint that defines the data and behaviour of an entity in the healthcare system.

Examples of classes include:

- Patient
- Doctor
- Appointment
- MedicalRecord
- Prescription
- HospitalStaff

For example, the Patient class may contain attributes such as patient ID, name, age, contact number, and medical history. It may also contain methods such as book appointment, view prescription, and update contact details. Classes provide a structured way to organize healthcare-related data and functions.

2. Object

An object is a specific instance of a class.

For example:

```
Patient patient1 = new Patient();
```

```
patient1.patientId = 101;
```

```
patient1.name = "Ravi";
```

```
patient1.age = 35;
```

Here, patient1 is an object of the Patient class.

Similarly, individual doctors, appointments, and medical records are represented as objects. Each object contains its own data while using the structure and behaviour defined by its class.

3. Encapsulation

Encapsulation means combining data and methods within a class and restricting direct access to sensitive information.

In a healthcare system, information such as medical history, diagnosis, prescriptions, and test results must be protected. These attributes can be declared as private and accessed only through authorized methods.

Encapsulation ensures that medical information cannot be directly modified by unauthorized users. For example, a patient may be allowed to view a medical record, but only a doctor may update the diagnosis.

Therefore, encapsulation improves data privacy, security, and accuracy.

4. Inheritance

Inheritance allows one class to acquire the common properties and methods of another class.

For example, a general class called Person may contain common attributes such as name, age, address, and contact number.

The classes Patient, Doctor, and Nurse can inherit these attributes.

Inheritance avoids repeating common code in multiple classes. It also makes it easier to add new healthcare roles such as pharmacist, laboratory technician, or receptionist.

5. Polymorphism

Polymorphism means that the same method can perform different actions depending on the object that uses it.

For example, different users may have different implementations of the method `displayDashboard()`.

When the method is called, the system displays the appropriate dashboard according to whether the user is a patient, doctor, or administrator.

Polymorphism makes the system flexible because one common interface can support different user behaviours.

Benefits of Object-Oriented Concepts

1. Scalability

Object-oriented design improves scalability by dividing the healthcare system into independent classes and modules.

New features such as telemedicine, online payment, pharmacy management, laboratory reports, and insurance processing can be added as separate classes without changing the entire system. Inheritance also allows new healthcare staff roles to be introduced easily by extending existing classes. Therefore, the system can grow according to increasing numbers of patients, doctors, hospitals, and services.

2. Security

Encapsulation protects confidential healthcare information by preventing direct access to sensitive attributes.

Access control can be implemented based on user roles. For example:

- Patients can view their own medical records.

- Doctors can update diagnosis and treatment details..

This reduces unauthorized access, accidental modification, and misuse of patient information.

3. Maintainability

Object-oriented systems are easier to maintain because each class has a specific responsibility.

For example, appointment-related changes can be made in the Appointment class without affecting the MedicalRecord class.

Code reuse through inheritance reduces duplication, while polymorphism makes it easier to modify or extend existing functions. Errors can also be identified and corrected within individual modules.

As a result, the software becomes easier to test, update, debug, and manage.

Conclusion

Object-oriented concepts provide an effective method for designing a Smart Healthcare System. Classes and objects represent real-world healthcare entities, encapsulation protects sensitive patient information, inheritance promotes code reuse, and polymorphism supports flexible behaviour.

Together, these concepts improve the scalability, security, flexibility, and maintainability of the healthcare system, making it suitable for modern hospitals and digital healthcare services.

2. Applying Encapsulation and Abstraction in Online Banking

An Online Banking System provides services such as fund transfer, account management, and transaction tracking.

Discuss the concepts of abstraction and encapsulation in object-oriented design. Compare their roles and analyze how they help in protecting sensitive data and simplifying system complexity.

Answer :

An Online Banking System provides services such as fund transfer, account management, balance enquiry, bill payment, and transaction tracking. Since banking systems handle

confidential customer and financial information, object-oriented concepts such as encapsulation and abstraction are important for ensuring security and reducing system complexity.

1. Encapsulation

Encapsulation is the process of combining data and the methods that operate on that data within a single class. It also restricts direct access to sensitive data by declaring attributes as private.

In an online banking system, information such as account balance, PIN, password, transaction history, and account number should not be directly accessed or modified by other parts of the program.

For example, the account balance and PIN are private. They can only be accessed through controlled methods such as `getBalance()`, `deposit()`, and `verifyPin()`.

Encapsulation prevents unauthorized users or objects from directly changing sensitive banking information.

2. Abstraction

Abstraction means hiding unnecessary implementation details and showing only the essential functions to the user.

A customer using an online banking application can select options such as:

- Transfer funds
- Check account balance
- Pay bills
- Download account statements
- Track transactions

The customer does not need to know how the system verifies the account, communicates with the bank server, encrypts the transaction, or updates the database. The customer interacts with a simple banking service interface, while the complex transaction-processing steps remain hidden.

Comparison of Encapsulation and Abstraction

Basis	Encapsulation	Abstraction
Meaning	Combines data and methods into one class	Hides complex implementation details
Main purpose	Protects data from unauthorized access	Simplifies interaction with the system
Focus	Internal security and controlled data access	Essential functions visible to users
Implementation	Private variables, getters, setters and access modifiers	Abstract classes, interfaces and simplified service methods
Banking example	Protecting account balance and PIN	Displaying a simple “Transfer Funds” option
Question answered	How is the data protected?	What service is provided?

Role in Protecting Sensitive Data

Encapsulation protects sensitive banking data by restricting direct access to customer details and account information.

For example:

- Customers cannot directly modify their account balance.
- A PIN can only be checked through an authentication method.
- Transaction records can only be updated through authorized banking operations.
- Account information can be accessed based on user permissions.

Validation can also be included inside methods. For instance, a withdrawal method can check whether the requested amount is valid and whether sufficient balance is available before completing the transaction.

Abstraction also contributes to security by hiding internal processes such as encryption methods, database queries, fraud-detection rules, and authentication mechanisms. Users only see the required banking operations and cannot directly interact with the internal components.

Role in Simplifying System Complexity

Online banking involves many complex processes, including customer authentication, beneficiary verification, transaction authorization, database updates, notification services, and security checks.

Abstraction hides these processes behind simple functions. For example, when the customer clicks “Transfer,” the system may perform several internal operations:

1. Verify login credentials.
2. Validate the beneficiary.
3. Check the available balance.
4. Generate a one-time password.
5. Process the transfer.
6. Update the account balance.
7. Record the transaction.
8. Send a confirmation message.

The customer only sees a simple fund-transfer screen instead of all these internal steps.

Encapsulation also reduces complexity by placing related data and functions inside separate classes such as Customer, BankAccount, Transaction, Beneficiary, and FundTransfer. Changes made in one class do not unnecessarily affect other parts of the system.

Benefits to the Online Banking System

- 1.Improved Security : Sensitive information is kept private and can only be accessed through authorized methods. This reduces unauthorized access, accidental modification, and misuse of customer data.
- 2.Reduced Complexity : Customers and developers interact with simple interfaces without needing to understand the complete internal working of the banking system.
- 3.Better Maintainability : Each class has a specific responsibility. For example, changes to transaction tracking can be made in the Transaction class without modifying the account-management module.

4.Data Validation : Encapsulated methods can validate all operations before changing account data. This prevents invalid deposits, unauthorized withdrawals, and incorrect transfers.

5.Easier System Extension: New services such as mobile recharge, loan payment, card management, and international transfer can be added without changing the entire system.

Conclusion

Encapsulation and abstraction are essential object-oriented concepts in an Online Banking System. Encapsulation protects confidential data by restricting direct access and allowing controlled operations through methods. Abstraction hides complex banking processes and presents only the necessary services to customers.

Although encapsulation mainly focuses on data protection and abstraction focuses on hiding complexity, both work together to create an online banking system that is secure, simple, reliable, scalable, and easy to maintain.

3. Modeling Object Relationships in a University Management System

A University Management System manages students, faculty, departments, and courses.

Analyze different object relationships such as association, aggregation, and composition. Explain how appropriate relationship modeling improves system flexibility, reusability, and maintainability.

Answer :

A University Management System manages students, faculty members, departments, courses, examinations, and academic records. In object-oriented design, these entities are represented as classes and objects. Relationships between objects describe how different parts of the system interact and depend on one another.

The main object relationships used in a University Management System are association, aggregation, and composition.

1. Association

Association represents a general relationship between two independent classes. The objects are connected, but neither object controls the existence of the other.

For example, a student enrolls in a course. A student can enrol in several courses, and a course can contain several students. Therefore, the relationship between Student and Course is a many-to-many association.

Other examples include:

- A faculty member teaches a course.
- A student attends a class.
- A student consults a faculty member.
- A faculty member supervises a student.
- A student participates in an examination.

Both objects can exist independently. A student continues to exist even if a course is discontinued, and a course can exist even if no student is currently enrolled.

In a UML class diagram, association is represented using a solid line between two classes.

Student — enrolls ————— Course

2. Aggregation

Aggregation represents a weak whole-part relationship. One object contains or groups other objects, but the contained objects can exist independently of the whole.

For example, a department contains several faculty members. Faculty members belong to a department, but they can continue to exist if the department is reorganized or removed. They may also be transferred to another department.

Other examples include:

- A university contains several departments.
- A department offers several courses.
- A faculty member handles multiple courses.
- A library contains several books.

- A department includes students and staff members.

In aggregation, the whole object does not completely control the lifecycle of the part object.

In a UML class diagram, aggregation is represented using a hollow diamond at the whole side.

Department ◇———— Faculty

Here, the department is the whole, while faculty members are its parts.

3. Composition

Composition represents a strong whole-part relationship. The part object depends completely on the whole object for its existence.

For example, a student academic record contains semester results. A semester result belongs to a particular student record and normally has no independent existence. If the student record is permanently deleted, the related semester results are also deleted.

Other examples include:

- A course contains course modules.
- An examination contains examination questions.
- A student record contains attendance entries.
- A timetable contains timetable slots.
- A student application contains submitted documents.

Composition is used when the part cannot meaningfully exist without the whole.

In a UML class diagram, composition is represented using a filled diamond at the whole side.

StudentRecord ◆———— SemesterResult

Comparison of Object Relationships

Basis	Association	Aggregation	Composition
-------	-------------	-------------	-------------

Meaning	General relationship between independent objects	Weak whole-part relationship	Strong whole-part relationship
Ownership	No ownership	Partial or weak ownership	Complete ownership
Dependency	Objects are independent	Part can exist without the whole	Part depends on the whole
Lifecycle	Separate lifecycles	Mostly separate lifecycles	Common lifecycle
UML symbol	Solid line	Hollow diamond	Filled diamond
Example	Student enrolls in Course	Department contains Faculty	StudentRecord contains SemesterResult
Effect of deleting the whole	Related object remains	Part normally remains	Part is also deleted

Combined Relationship Model

A University Management System may use all three relationships together.

University ◇———— Department

Department ◇———— Faculty

Faculty ————— Course

Student ————— Course

StudentRecord ◆———— SemesterResult

Course ◆———— CourseModule

In this model:

- A university aggregates several departments.
- A department aggregates faculty members.
- Faculty members are associated with courses.

- Students are associated with courses through enrolment.
- A student record is composed of semester results.
- A course is composed of course modules.

Improvement in System Flexibility

Appropriate relationship modeling improves flexibility because objects can be changed or reassigned based on their level of dependency.

For example, the association between faculty and course allows a course to be assigned to another faculty member without deleting the course. Similarly, aggregation allows a faculty member to be transferred from one department to another.

New entities such as visiting faculty, research scholars, online courses, and certificate programmes can also be added without redesigning the complete system.

Therefore, the system can easily adapt to changes in university structure, academic rules, and course requirements.

Improvement in Reusability

Relationship modeling makes classes independent and reusable across different modules.

For example, the Course class can be reused in:

- Student enrolment
- Faculty allocation
- Timetable preparation
- Attendance management
- Examination management
- Result processing

Similarly, the Faculty class can be reused in teaching, research supervision, student mentoring, and department management.

This reduces code duplication and prevents the same data from being created repeatedly in different modules.

Improvement in Maintainability

Clearly defined relationships make the system easier to understand, test, and modify.

For example:

- Changes to student enrolment mainly affect the relationship between students and courses.
- Changes to department structure mainly affect aggregation relationships.
- Changes to academic results mainly affect the composition between student records and semester results.

This separation prevents modifications in one module from unnecessarily affecting unrelated modules.

Proper relationship modeling also ensures correct deletion behaviour. For example, deleting a department should not automatically delete faculty members because they may be transferred. However, deleting a student record may also remove its semester results because those results depend on that record.

Conclusion

Association, aggregation, and composition are important object relationships in the design of a University Management System. Association represents interaction between independent objects, aggregation represents a weak whole-part relationship, and composition represents a strong dependency between the whole and its parts.

4. Importance of SDLC in E-Commerce Development

An E-Commerce Platform is developed to support product management, online payments, and order tracking.

Explain the phases of Software Development Life Cycle (SDLC). Analyze how each phase contributes to software quality, reliability, and customer satisfaction.

Answer :

An E-Commerce Platform supports functions such as product management, customer registration, online payments, shopping carts, order processing, delivery tracking, and customer support. Developing such a system requires a structured process to ensure that it is secure, reliable, user-friendly, and capable of handling large numbers of users.

The Software Development Life Cycle (SDLC) is a systematic process used to plan, design, develop, test, deploy, and maintain software. The main phases of SDLC are requirement analysis, planning, system design, implementation, testing, deployment, and maintenance.

1. Requirement Analysis

Requirement analysis is the process of identifying and documenting what the e-commerce platform must do.

The development team collects requirements from customers, business owners, administrators, sellers, payment providers, and delivery partners.

Important requirements may include:

- Customer registration and login
 - Product search and filtering
 - Shopping-cart management
 - Secure online payment
 - Order placement and cancellation
 - Inventory management
 - Delivery tracking
 - Product reviews and ratings
 - Return and refund processing
 - Administrative reporting
-
- Contribution to Software Quality : Clearly defined requirements ensure that the correct features are developed. They reduce misunderstandings and prevent unnecessary functionality.
 - Contribution to Reliability : Security, performance, availability, and transaction requirements are identified before development begins. This helps the system handle payments and orders accurately.

- Contribution to Customer Satisfaction : Understanding customer needs helps create features that are convenient, useful, and easy to operate.

2. Planning and Feasibility Study

During the planning phase, the project scope, budget, schedule, resources, technologies, risks, and development approach are determined.

The feasibility study examines whether the project is technically, economically, legally, operationally, and financially possible.

For an e-commerce platform, planning may include:

- Selecting suitable web and mobile technologies
 - Estimating development costs
 - Choosing payment gateways
 - Determining server and storage requirements
 - Planning security measures
 - Identifying legal and privacy requirements
 - Estimating the number of expected users
 - Preparing a project schedule
- Contribution to Software Quality : Proper planning ensures that skilled developers, suitable technologies, and sufficient resources are available.
 - Contribution to Reliability : Risks such as server failure, payment errors, cyberattacks, and project delays can be identified and controlled in advance.
 - Contribution to Customer Satisfaction : A realistic plan helps deliver the platform on time with the promised features and performance.

3. System Design

The design phase converts the requirements into a technical structure for the system.

It includes:

- System architecture
- Database design

- User-interface design
- Product-catalogue structure
- Payment-processing workflow
- Order-processing workflow
- Security and access control
- Integration with delivery services
- Data backup and recovery plans

For example, the design defines how customers, products, carts, orders, payments, and delivery information are connected.

- Contribution to Software Quality : A clear design improves consistency, modularity, usability, and scalability.
- Contribution to Reliability : The system can be designed with secure payment processing, database backups, error handling, and recovery mechanisms.
- Contribution to Customer Satisfaction : A well-designed interface makes it easier for customers to search for products, complete payments, and track orders.

4. Implementation or Development

During implementation, developers convert the system design into a working application.

Different modules are developed, such as:

- User-account management
- Product management
- Shopping cart
- Payment processing
- Inventory control
- Order management
- Delivery tracking
- Notifications
- Reviews and ratings
- Administration panel

The modules are then integrated to form the complete e-commerce platform.

- Contribution to Software Quality : Coding standards, documentation, modular development, and code reviews help produce accurate and maintainable software.
- Contribution to Reliability : Validation, exception handling, secure coding, and transaction controls reduce failures and incorrect operations.
- Contribution to Customer Satisfaction : Correct implementation ensures that users can perform activities such as adding products to the cart, making payments, and tracking orders without difficulty.

5. Testing

Testing verifies whether the developed platform works according to its requirements and whether it is free from serious errors.

Important forms of testing include:

- Unit testing: Tests individual functions and modules.
 - Integration testing: Checks communication between modules.
 - System testing: Tests the complete e-commerce platform.
 - Security testing: Identifies vulnerabilities and unauthorized access.
 - Performance testing: Checks response time and ability to handle many users.
 - Usability testing: Evaluates whether the platform is easy to use.
 - Payment testing: Ensures that payment transactions are processed correctly.
 - User acceptance testing: Confirms that the platform meets business and customer needs.
-
- Contribution to Software Quality : Testing identifies defects before the software reaches customers. It verifies that all features work correctly.
 - Contribution to Reliability : Testing ensures that orders, payments, refunds, inventory updates, and delivery tracking function consistently under different conditions.
 - Contribution to Customer Satisfaction : A properly tested system provides faster performance, fewer errors, secure transactions, and a smooth shopping experience.

6. Deployment

Deployment is the process of releasing the e-commerce platform for actual use.

Activities may include:

- Installing the application on servers
- Configuring databases
- Connecting the payment gateway
- Importing product information
- Setting up security certificates
- Configuring backup systems
- Publishing mobile or web applications
- Training administrators and support teams

The platform may initially be released to a limited group of users before being made available to everyone.

- Contribution to Software Quality : A controlled deployment process ensures that the software is installed and configured correctly.
- Contribution to Reliability :Backup plans, server monitoring, and rollback procedures reduce the risk of service interruptions.
- Contribution to Customer Satisfaction : A smooth launch allows customers to access the platform without major technical problems.

7. Maintenance

Maintenance begins after the e-commerce platform is deployed. It includes correcting errors, improving performance, updating security, and adding new features.

Common maintenance activities include:

- Fixing software defects
- Updating product and payment services
- Improving website speed
- Installing security patches
- Monitoring server performance
- Updating mobile applications
- Adding new payment options
- Improving search and recommendation features
- Supporting new devices and browsers

- Responding to customer feedback
- Contribution to Software Quality : Regular updates improve the platform and prevent it from becoming outdated.
- Contribution to Reliability : Continuous monitoring, backups, security updates, and defect correction keep the system stable and available.
- Contribution to Customer Satisfaction :Customer feedback can be used to improve navigation, payment methods, delivery tracking, and support services.

Contribution of SDLC Phases

SDLC Phase	Main Activity	Contribution to Quality	Contribution to Reliability	Contribution to Customer Satisfaction
Requirement analysis	Identifying system needs	Ensures correct features	Defines performance and security needs	Reflects customer expectations
Planning	Estimating resources, time, and risks	Supports organized development	Reduces technical and project risks	Encourages timely delivery
Design	Preparing architecture and interfaces	Creates a scalable structure	Supports security and recovery	Improves user experience
Implementation	Developing system modules	Produces functional software	Adds validation and error handling	Provides required services
Testing	Detecting and correcting defects	Improves correctness	Ensures stable operations	Reduces failures and frustration
Deployment	Releasing the system	Ensures proper configuration	Supports availability and rollback	Provides a smooth system launch

Maintenance	Updating and improving the system	Maintains long-term quality	Prevents failures and security issues	Responds to changing customer needs
--------------------	-----------------------------------	-----------------------------	---------------------------------------	-------------------------------------

Overall Importance of SDLC in E-Commerce Development

- **Improved Software Quality** : SDLC ensures that the platform is developed using clear requirements, proper design, coding standards, systematic testing, and continuous improvement.
- **Secure Online Transactions** : Security requirements are considered throughout the development process. This helps protect customer accounts, payment details, and personal information.
- **Higher Reliability** : Testing, monitoring, backup, and recovery measures help the platform remain available and function correctly, even when many customers use it at the same time.
- **Better Customer Experience** : The SDLC process focuses on customer requirements, interface design, usability testing, and feedback. This produces a convenient and user-friendly shopping experience.
- **Reduced Development Risk** : Risks are identified during planning, design, and testing. This reduces unexpected costs, project delays, and system failures.
- **Easier Maintenance** : The system is divided into separate modules, making it easier to fix errors or update one function without affecting the entire platform.
- **Improved Scalability** : The system can be designed to support increasing numbers of customers, products, sellers, orders, and transactions.

Conclusion

The Software Development Life Cycle provides a structured approach for developing an e-commerce platform. Requirement analysis identifies customer and business needs, planning controls resources and risks, design creates the system structure, implementation develops the required functions, testing detects errors, deployment releases the platform, and maintenance ensures continuous improvement.

Each SDLC phase contributes to software quality, system reliability, security, and customer satisfaction. Therefore, following the SDLC is essential for developing a successful, secure, scalable, and user-friendly e-commerce platform.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness